



- [Atom](#)
- [RSS](#)
- [Blogs](#)
- [Alpha Scientist Consulting](#)

Stock Prediction with ML: Walk-forward Modeling

Author: Chad Gray

Wed 18 July 2018

Key Takeaways:

- Traditional methods of validation and cross-validation are problematic for time series prediction problems
- The solution is to use a "walk-forward" approach which incorporates new information as it becomes available.
- This approach gives us a more realistic view of how effective our model would truly have been in the past, and helps to avoid the overfitting trap.
- It's often important to exclude data which is too far in the past using a rolling window.

Introduction

This is the fourth post in my series on transforming data into alpha. If you haven't yet see the [framework overview](#), [feature engineering guide](#), or [feature selection guide](#), please take a minute to read those first. This post will build on the framework and philosophy presented there.

This post is going to delve into the mechanics of *walk-forward modeling* which is, in my view, the most robust way to train and apply machine learning models in inherently sequential domains like finance. If you'd like to replicate and experiment with the below code, *you can download the source notebook for this post by right-clicking on the below button and choosing "save link as"*

ipynb notebook

The domain of market prediction presents several unique challenges for machine learning practitioners which do not exist in spam detection, natural language processing, image recognition, or other common areas of machine learning success, including:

- Low signal-to-noise ratio
- Non-stationarity (aka regime switching)
- Market adaptation (aka reflexivity)

The combination of these challenges make it difficult to train and tune models which *generalize* to the future, where it really matters. This, in my view, has led many to conclude that markets cannot be predicted with machine learning techniques.

This is a flawed conclusion. Instead, we should conclude that it's imperative to follow a modeling process which (1) avoids the trap of overfitting models to noise, (2) takes into account the sequential dimension of time series data, and (3) allows us to get an accurate view about how various models would have performed in the past *given the information we would have had at that time*.

"But wait!" I hear you saying, "Overfitting is not unique to market prediction!". True, but the commonly used techniques of *train-test split* and *cross-validation* each have major flaws when applied to an inherently sequential set of financial time series data.

The overriding objective of the methods described here is to **overcome the issues inherent in traditional cross validation approaches**.

Issues with typical approaches

Train-Test Split: One frequently recommended approach in finance ML tutorials is to split the data (commonly, by time) and to train on the first chunk of data and test out-of-sample on the second. This is not wrong per se, but creates two problems:

1. Only a fraction of data can be used to generate out-of-sample performance of the model. If you split 80/20, then you only get results for 2 of every 10 years of data. Yuck.
2. You're implicitly biased towards the most recent period. Since we're likely going to use the older data to train and newer data to test (for good reason...) you are implicitly searching for a model which was in favor during the most recent couple of years. Maybe last year is a good approximation for next year, but maybe it's not. Maybe 2008 would hold some important lessons for us to evaluate, but these would be lost to us.

You might consider train/test split with random (or non-random) sampling but this creates other fairly obvious issues with peeking into the future.

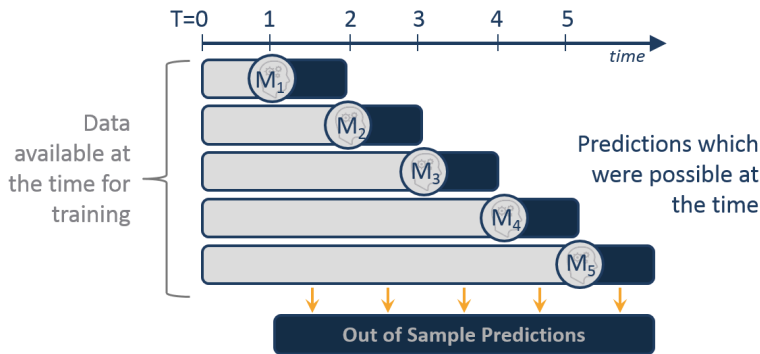
Cross Validation: In normal ML usage, cross-validation methods, such as the [K-fold cross validation](#), provide an elegant way to have our digital cake and eat it too. However, with inherently sequential datasets, it's really not kosher to train a model on 2016 and 2018 to predict 2017.

Another common method, [leave one out cross-validation](#), creates even more extreme peeking into the future. *These issues are especially intolerable if your model predicts values multiple bars in the future*, which results in your models being trained on data which significantly overlaps with your test data - a subtle but deadly form of data spillover.

The Solution: Walk-forward Train/Test

But we are not stuck with either of these problematic approaches. The better - and I think much more intuitive - approach is to simulate models in a "walk-forward" sequence, periodically re-training the model to incorporate all data available at that point in time.

The below picture is worth at least 1000 words. Say we have six years of data to train + test. At the end of the first year ($T=1$) we could have built model M_1 , which was trained on the first year's data (the grey bar). After the 5th year, we could use up to five years of data to train model M_5 .



Getting Started

As in prior posts, we'll begin by fetching some real stock price data and then generating from it a synthetic dataset which will make it easier to follow the principles of this tutorial. Please refer to the previous post [Feature Selection](#) for further explanation of the reasoning and mechanics of creating synthetic data.

```
In [1]: from IPython.core.display import HTML, Image
import sys
sys.path.append('/anaconda/')
import config

import numpy as np
import pandas as pd
pd.core.common.is_list_like = pd.api.types.is_list_like
# Needed for this issue: https://github.com/pydata/pandas-datareader/issues/534
import pandas_datareader.data as web
%matplotlib inline

def get_symbols(symbols, data_source, begin_date=None, end_date=None):
    out = pd.DataFrame()
    for symbol in symbols:
        df = web.DataReader(symbol, data_source, begin_date, end_date)[['AdjOpen', 'AdjHigh', 'AdjLow', 'AdjClose', 'AdjVolume']].reset_index()
        df.columns = ['date', 'open', 'high', 'low', 'close', 'volume'] #my convention: always lowercase
        df['symbol'] = symbol # add a new column which contains the symbol so we can keep multiple symbols in the same dataframe
        df = df.set_index(['date', 'symbol'])
        out = pd.concat([out, df], axis=0) #stacks on top of previously collected data
    return out.sort_index()

prices = get_symbols(['AAPL', 'CSCO', 'AMZN', 'YHOO', 'MSFT'], data_source='quandl', begin_date='2012-01-01', end_date=None)
# note, we're only using real price data to get an accurate date/symbol index set.
```

Similar to the [previous post](#), we will generate some random data to serve as features *and then derive* an outcome (response) variable from the features (to ensure that our made-up features will have ability to partially predict the target variable).

However, this go around we'll add two non-random patterns to the data. First, we will add some amount of "memory" (autoregression) to the white noise with a short function called `add_memory()`. This will make the data somewhat less random and more similar to actual markets - which I think we can all agree is not a purely gaussian random walk...

Second, we'll add trend to the importance of each feature to the target variable to simulate the reality that various features - and the market factors they represent - move in and out of favor over time (aka "regime shift").

- Feature f_{01} is becoming more important
- Feature f_{02} is becoming less important
- Feature f_{03} is oscillating between very important and unimportant (using a sine wave pattern)
- Feature f_{04} is not changing

```
In [2]: num_obs = prices.close.count()

def add_memory(s,n_days=50,memory_strength=0.1):
    ''' adds autoregressive behavior to series of data'''
    add_ewm = lambda x: (1-memory_strength)*x + memory_strength*x.ewm(n_days).mean()
    out = s.groupby(level='symbol').apply(add_ewm)
    return out

# generate feature data
f01 = pd.Series(np.random.randn(num_obs),index=prices.index)
f01 = add_memory(f01,10,0.1)
f02 = pd.Series(np.random.randn(num_obs),index=prices.index)
f02 = add_memory(f02,10,0.1)
f03 = pd.Series(np.random.randn(num_obs),index=prices.index)
f03 = add_memory(f03,10,0.1)
f04 = pd.Series(np.random.randn(num_obs),index=prices.index)
f04 = f04 # no memory

## now, create response variable such that it is related to features
# f01 becomes increasingly important, f02 becomes decreasingly important,
# f03 oscillates in importance, f04 is stationary, finally a noise component is added

outcome = f01 * np.linspace(0.5,1.5,num_obs) + \
          f02 * np.linspace(1.5,0.5,num_obs) + \
          f03 * pd.Series(np.sin(2*np.pi*np.linspace(0,1,num_obs)*2)+1,index=f03.index) + \
          f04 + \
          np.random.randn(num_obs) * 3
outcome.name = 'outcome'
```

So, we've now got four features which contain random and non-random elements, and we have an outcome target variable which can be partially explained by these features.

Let's run a quick linear model to verify that this worked as expected:

```
In [3]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
features = pd.concat([f01,f02,f03,f04],axis=1)
features.columns = ['f01','f02','f03','f04']
model.fit(X=features,y=outcome)
print('RSQ: '+str(model.score(X=features,y=outcome)))
print('Regression Coefficients: '+str(model.coef_))

RSQ: 0.26872811401332664
Regression Coefficients: [0.97587665 1.04585218 1.01461189 0.99374324]
```

Indeed. A simple, linear model run on the entire dataset shows a remarkable (for this domain) RSQ of >0.25 and approximately equal coefficients for each of the features, suggesting that they're all contributing strongly to the model.

However, this overlooks the very important reality that our data is non-stationary and what matters is how the model will *generalize* to future, unseen data.

To illustrate, I'll re-run the same model after splitting the data into train/test sets, with the first 80% used to train and the last 20% used to test.

```
In [4]: split_point = int(0.80*len(outcome))

X_train = features.iloc[:split_point,:]
y_train = outcome.iloc[:split_point]
X_test = features.iloc[split_point:,:]
y_test = outcome.iloc[split_point:]

model = LinearRegression()
model.fit(X=X_train,y=y_train)

print('RSQ in sample: '+str(model.score(X=X_train,y=y_train)))
print('RSQ out of sample: '+str(model.score(X=X_test,y=y_test)))
print('Regression Coefficients: '+str(model.coef_))

RSQ in sample: 0.29476591841646305
RSQ out of sample: 0.13667733307962338
Regression Coefficients: [0.8361922 1.15876943 1.20372473 1.01599709]
```

Here, the results tell a different story. We see that the model's RSQ on the unseen test data degraded to only about half of the value on the training data. Even if we were still satisfied with the degraded RSQ value, we would carry concern that perhaps the next period of time the models would degrade even further.

This is where our walk-forward process comes in.

A better approach...



Here, I'll outline the basic design pattern for a robust approach to walk-forward modeling. This is simplified. In real-world usage, we'd likely add many enhancements but this will illustrate the concept.

Training models

The first step of the walk-forward approach is to train our model *using only the data available at any given point in time*. Theoretically, we have new data every bar so could retrain the model daily if we wished. However, for simplicity we'll retrain much less frequently - at the end of each calendar year.

We'll first create a list of timestamps for all end-of-year dates in our dataset. These will become our days on which to retrain models. Then, on each of the recalculation dates, we'll follow a simple procedure:

1. slice out only the data which has happened up until that point
2. train a model with that data
3. save a copy of that fitted model object in a pandas Series called `models` which is indexed by the recalculation date.

We could just as easily have used a dict or other data structure, but I find that pandas Series works well for this purpose. Pandas is not just for numbers, after all...

```
In [5]: recal_dates = features.resample('Q', level='date').mean().index.values[:-1]
        #print('recal_dates:')
        #print(recal_dates)
        #print()

        models = pd.Series(index=recal_dates)
        for date in recal_dates:
            X_train = features.xs(slice(None, date), level='date', drop_level=False)
            y_train = outcome.xs(slice(None, date), level='date', drop_level=False)
            model = LinearRegression()
            model.fit(X_train, y_train)
            models.loc[date] = model

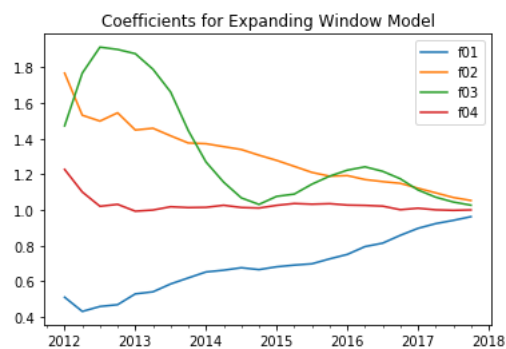
            print("Training on the first {} records, through {}"\
                  .format(len(y_train), y_train.index.get_level_values('date').max()))
            #print("Coefficients: {}".format(model.coef_))
            #print()
```

```
Training on the first 310 records, through 2012-03-30 00:00:00
Training on the first 625 records, through 2012-06-29 00:00:00
Training on the first 940 records, through 2012-09-28 00:00:00
Training on the first 1250 records, through 2012-12-31 00:00:00
Training on the first 1550 records, through 2013-03-28 00:00:00
Training on the first 1870 records, through 2013-06-28 00:00:00
Training on the first 2190 records, through 2013-09-30 00:00:00
Training on the first 2510 records, through 2013-12-31 00:00:00
Training on the first 2815 records, through 2014-03-31 00:00:00
Training on the first 3130 records, through 2014-06-30 00:00:00
Training on the first 3450 records, through 2014-09-30 00:00:00
Training on the first 3770 records, through 2014-12-31 00:00:00
Training on the first 4075 records, through 2015-03-31 00:00:00
Training on the first 4390 records, through 2015-06-30 00:00:00
Training on the first 4710 records, through 2015-09-30 00:00:00
Training on the first 5030 records, through 2015-12-31 00:00:00
Training on the first 5335 records, through 2016-03-31 00:00:00
Training on the first 5655 records, through 2016-06-30 00:00:00
Training on the first 5975 records, through 2016-09-30 00:00:00
Training on the first 6290 records, through 2016-12-30 00:00:00
Training on the first 6600 records, through 2017-03-31 00:00:00
Training on the first 6905 records, through 2017-06-30 00:00:00
Training on the first 7155 records, through 2017-09-29 00:00:00
Training on the first 7403 records, through 2017-12-29 00:00:00
```

To visualize how the model is changing over time, I'll create a simple plotting function of coefficients:

```
In [6]: def extract_coefs(models):
        coefs = pd.DataFrame()
        for i,model in enumerate(models):
            model_coefs = pd.Series(model.coef_,index=['f01','f02','f03','f04']) #extract coefficients for model
            model_coefs.name = models.index[i] # name it with the recal date
            coefs = pd.concat([coefs,model_coefs],axis=1)
        return coefs.T
        extract_coefs(models).plot(title='Coefficients for Expanding Window Model')
```

Out[6]: <matplotlib.axes._subplots.AxesSubplot at 0xc6e68d0>



Notice a few things about how the model's coefficients change over time:

- The first feature gets increasing weight as it becomes a more important explanatory factor in the target variable,
- the second feature does just the opposite, as it should,
- the third feature oscillates before converging on a relatively lower endpoint, and
- the fourth feature is mostly unchanged

But, in a world of constantly shifting regimes, we can do better.

We know that, by the end of the dataset, f01 is actually much superior to f02 however they end at about equal weights. This is because in our final model building instance, we are using *all* of our data to train, and this entire set of data correctly shows that *on average* f02 is as useful as f01, ignoring that f02's best days are long past.

```
In [7]: recalc_dates = features.resample('Q', level='date').mean().index.values[:-1]

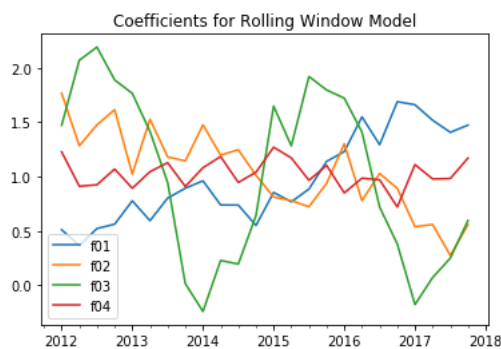
models = pd.Series(index=recalc_dates)
for date in recal_dates:
    X_train = features.xs(slice(date-pd.Timedelta('90 days'), date), level='date', drop_level=False)
    y_train = outcome.xs(slice(date-pd.Timedelta('90 days'), date), level='date', drop_level=False)
    model = LinearRegression()
    model.fit(X_train, y_train)
    models.loc[date] = model

    print("Training on the most recent {} records".format(len(y_train)))
    #print("Coefficients: {}".format(model.coef_))

extract_coefs(models).plot(title='Coefficients for Rolling Window Model')
```

```
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 305 records
Training on the most recent 305 records
Training on the most recent 320 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 305 records
Training on the most recent 320 records
Training on the most recent 315 records
Training on the most recent 315 records
Training on the most recent 310 records
Training on the most recent 305 records
Training on the most recent 250 records
Training on the most recent 248 records
```

```
Out[7]: <matplotlib.axes._subplots.AxesSubplot at 0xa611a58>
```



This approach of only considering the 90 days' data prior to each model re-training has had a big impact. We now see that f_{01} has overtaken f_{02} in significance by the end of the time period, as we'd hope. The on-again, off-again feature f_{03} has risen and fallen in prominence over time, again just as we'd hope.

Using models

The second stage of the process is *using* these walk-forward models. The process is similar and equally simple.

First, we create two arrays, `begin_dates` and `end_dates`, which contain the dates on which each model is used. For instance, for the first model ($i=0$), we will apply this model to features beginning on the date we trained the model (`recalc_date`) until the day the next model is trained. The `end_dates` array therefore drops the 0th element and appends to the end a far-off date (in the year 2099).

We can equally easily use this pattern to apply the models to true out of sample features. As long as it's not later than the year 2099, we'll simply apply the latest and greatest model we have.

```
In [8]: begin_dates = models.index
end_dates = models.index[1:].append(pd.to_datetime(['2099-12-31']))

predictions = pd.Series(index=features.index)

for i,model in enumerate(models): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i],end_dates[i]), level='date', drop_level=False)
    p = pd.Series(model.predict(X), index=X.index)
    predictions.loc[X.index] = p

predictions.shape
```

```
Out[8]: (7639,)
```

So, this looped through each of the models we had trained at various points in time and, for each, used that model to predict the period of time until the next model became available. It runs!

Now we can confirm that the rolling model is, in fact, better than the "expanding" model by making predictions with each and comparing to truth.

```
In [9]: models_expanding_window = pd.Series(index=recalc_dates)
for date in recalc_dates:
    X_train = features.xs(slice(None,date),level='date',drop_level=False)
    y_train = outcome.xs(slice(None,date),level='date',drop_level=False)
    model = LinearRegression()
    model.fit(X_train,y_train)
    models_expanding_window.loc[date] = model

models_rolling_window = pd.Series(index=recalc_dates)
for date in recalc_dates:
    X_train = features.xs(slice(date-pd.Timedelta('90 days'),date),level='date',drop_level=False)
    y_train = outcome.xs(slice(date-pd.Timedelta('90 days'),date),level='date',drop_level=False)
    model = LinearRegression()
    model.fit(X_train,y_train)
    models_rolling_window.loc[date] = model

begin_dates = models.index
end_dates = models.index[1:].append(pd.to_datetime(['2099-12-31']))

predictions_expanding_window = pd.Series(index=features.index)
for i,model in enumerate(models_expanding_window): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i],end_dates[i]),level='date',drop_level=False)
    p = pd.Series(model.predict(X),index=X.index)
    predictions_expanding_window.loc[X.index] = p

predictions_rolling_window = pd.Series(index=features.index)
for i,model in enumerate(models_rolling_window): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i],end_dates[i]),level='date',drop_level=False)
    p = pd.Series(model.predict(X),index=X.index)
    predictions_rolling_window.loc[X.index] = p
```

```
In [10]: from sklearn.metrics import r2_score

common_idx = outcome.dropna().index.intersection(predictions_expanding_window.dropna().index)
rsq_expanding = r2_score(y_true = outcome[common_idx],y_pred=predictions_expanding_window[common_idx])
rsq_rolling = r2_score(y_true = outcome[common_idx],y_pred=predictions_rolling_window[common_idx])

print("Expanding Window RSQ: {}".format(round(rsq_expanding,3)))
print("Rolling Window RSQ: {}".format(round(rsq_rolling,3)))

Expanding Window RSQ: 0.256
Rolling Window RSQ: 0.296
```

Great! All of that work added about 3 points to the RSQ, which is certainly worth the effort.

Avoiding complexity

The value of this walk-forward methodology is greatest when it helps you to avoid the scourge of overfitting. The linear regression model used up to this point is relatively resistant to overfit, since it has few parameters.

Let's say that, instead of a simple linear regression, we used the much more overfit-prone DecisionTree, which has tendency to "memorize the past" rather than to recognize patterns in it.

```
In [11]: from sklearn.tree import DecisionTreeRegressor

split_point = int(0.80*len(outcome))

X_train = features.iloc[:split_point,:]
y_train = outcome.iloc[:split_point]
X_test = features.iloc[split_point:,:]
y_test = outcome.iloc[split_point:]

model = DecisionTreeRegressor(max_depth=3)
model.fit(X=X_train,y=y_train)

print('RSQ in sample: '+str(round(model.score(X=X_train,y=y_train),3)))
print('RSQ out of sample: '+str(round(model.score(X=X_test,y=y_test),3)))

RSQ in sample: 0.179
RSQ out of sample: 0.008
```

This leads to overfit disaster! Now, we'll do the same with our walk-forward framework:

```
In [12]: recalculated_dates = features.resample('Q', level='date').mean().index.values[:-1]

models_rolling_window = pd.Series(index=recalculated_dates)
for date in recalculated_dates:
    X_train = features.xs(slice(date-pd.Timedelta('365 days'), date), level='date', drop_level=False)
    y_train = outcome.xs(slice(date-pd.Timedelta('365 days'), date), level='date', drop_level=False)
    model = DecisionTreeRegressor(max_depth=3)
    model.fit(X_train, y_train)
    models_rolling_window.loc[date] = model

predictions_rolling_window = pd.Series(index=features.index)
for i, model in enumerate(models_rolling_window): #Loop thru each models object in collection
    X = features.xs(slice(begin_dates[i], end_dates[i]), level='date', drop_level=False)
    p = pd.Series(model.predict(X), index=X.index)
    predictions_rolling_window.loc[X.index] = p

common_idx = y_test.dropna().index.intersection(predictions_rolling_window.dropna().index)
rsq_rolling = r2_score(y_true = y_test[common_idx], y_pred=predictions_rolling_window[common_idx])
print("RSQ out of sample (rolling): {}".format(round(rsq_rolling, 3)))

RSQ out of sample (rolling): 0.108
```

Much, much better. While the RSQ result ain't great (and actually worse than the linear regression model), that's much preferred to being fooled by an extraordinarily overfitted result like above. We could have guessed that DecisionTree would be a silly model type to try in this situation but it illustrates the point.

Summary

That, in a nutshell, is the walk-forward modeling framework. The key points to keep in mind are:

- Traditional methods of validation and cross-validation are problematic for time series prediction problems
- The solution is to use a "walk-forward" approach which incorporates new information as it becomes available.
- This approach gives us a more realistic view of how effective our model would truly have been in the past, and helps to avoid the overfitting trap.
- In addition to excluding any future data from the training process, it's often important to exclude data which is too far in the past.

In the next post of this series, I will present a framework for scoring and comparing models which is more complete and useful than generic machine learning methods.

One last thing...

If you've found this post useful, please follow [@data2alpha](#) on twitter and forward to a friend or colleague who may also find this topic interesting.

Posted by Chad Gray Wed 18 July 2018 [Blog tutorial, modeling](#)

Post

Comments

ALSO ON ALPHASCIENTIST.COM

Stock Prediction with ML: Feature ...

8 years ago • 17 comments

If you haven't yet seen the first post for this series, please take a minute to ...

ML for Stock Market Prediction (part 1): ...

8 years ago • 4 comments

About this tutorial¶ This is the first in a six-part series on the mechanics of ...

Our Own Worst Enemy — The ...

8 years ago • 5 comments

Key Takeaways¶ ETF investors appear to be consistently bad at timing ...

Reddit for Fun and Profit [part 2] — ...

5 years ago • 1 comment

In the prior post Tracking Posts on WallStreetBets - Part I, we demonstrated ...

Stock Prediction with ML: Data ...

8 years ago • 10 comments

About this tutorial¶ This is the first in a six-part series on the mechanics of ...

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name



2

Share

Best Newest Oldest**Robert Morris**

7 years ago edited

Really great and easy to follow tutorial on walk forward validation. You've presented a much simpler and useful method the the TimeSeriesSplit library provided in sklearn, in my opinion Thank you for sharing.

3 0 Reply

**Alexander Rios**

5 years ago

Hi Chad, would it make sense to implement a walk forward matrix to select the model?

I mean an array as described in the following link:

<https://tradingtact.com/dev...>

0 0 Reply

CP

Chris Pa

7 years ago

Hej Chad,

I really appreciate your series as I find it very useful and well-explained. I have got one question about the walk-forward modeling. For my understanding it's main purpose is to re-optimize the values after each "year" so to say.

I'm using an ANN to train my model and I thought this ANN is already doing that kind of re-optimization after each batch. So the question is: Does it make sense to use the walk-forward modeling using an artificial neural network at all?

Much appreciation from Germany,
Chris

0 0 Reply

A

Arbitrage- Technologies

8 years ago

Hi Chad,

thx for all these part on machine learning.

i am convinced that we can do much moore better than forward walk.

from my small experience in ml there is a very thin frontier between overfitting and robust leanring, which lies only in training set.

i thinks that the best path is to asses the distributions of your features, as you have been doing in other post, then fit them to the best distrib.

once you got the parameters of the distrib of each features, you just simulates a huge huge number of path and you train on that!

may be you have tried it in the past...

the overall question after that is how to build a real stock simulator engine which stick to reality...

0 0 Reply

**alphascientist** Mod

8 years ago

Arbitrage- Technologies

... And one common mistake I see made (not exactly related to this but sort of...) is using cross validation of randomly shuffled data on models trained for X days ahead prediction, thinking that you've estimated out of sample performance.

In other words, take 3000 days of data, each training models for a 50 day ahead horizon. Split that into 10 CV splits of 300 days each. Best case, if you keep sequential days together (days 1 to 300, 301 to 600 etc...) you are actually peeking into your held-out fold with each CV iteration. When holding out days 301 to 600 for testing, the feature values and outcome values for day 300 and day 301 are almost identical (since the target variable is 49/50ths overlapped). You're unintentionally peeking somewhat for 49 of the 300 days in each test fold.

Even worse, if randomly shuffled (fold 1 is days 1,19,22,38, ...) then there's heavy peeking for every single data point. Hopefully this makes sense w/o diagrams!

I mention this in conjunction with walk-forward because it's an example of how you can fool yourself into thinking you've built an exceptional model that backtests very well if using traditional ML techniques like K-fold CV. If you do whatever you do *inside of* a walk forward

framework, you're automatically guaranteed not to peek.

o o Reply ↗



alphascientist Mod → Arbitrage- Technologies

8 years ago

Hi JC, I think I understand what you're saying: develop huge amounts of synthetic data from the estimated distributions of each feature. I assume this would include modeling the distribution of the feature value itself as well as a distribution of the covariance of each feature with each other feature, since that's maybe the most critical parameter to get right if you want to successfully use those features in combination for prediction.

I haven't gone down this road, but know others have - and it seems to make sense if (a) you have confidence that you can get the distributions right and (b) you use algos that need that much data (e.g., complex DL networks).

Even if doing this, you would still need to walk-forward in the sense that you would want to simulate what your models would have looked like a year ago, or two years ago, or three years ago based on what you would have estimated the distributions to be *at that time* based on history to that point.

Walk forward is really just a way of forcing yourself to be honest about whether a given approach would have been successful in the past. Force yourself to retrain your models - using whatever approach - based on data which would have been available to you at a given point in time. Use predictions from those models to see whether you've found a durable approach or just a hindsight bias approach.

The flip side of this coin is that whatever model you trade live can benefit from all data which has happened in all of history, since the WF model you train today has access to all data which happened prior to today. If you choose to do monte carlo, synthetic data (as you suggest), or plain old actual history, you still make use of all data up to this point.

- Chad

o o Reply ↗



顏聰文

8 years ago

Hi Chad, have you tried using other Neural Network models, such as RNN, LSTM, or A3C? Is the performance better than a linear regression model? Thanks.

o o Reply ↗



alphascientist Mod → 顏聰文

8 years ago

I've dabbled (fully connected networks, LSTMs) and have generally found those models to be less robust out of sample than those limited by linearity constraints. That's not to say it can't be successfully done - and I haven't given up on trying.

I do think the right approach is to build a very simple model as baseline, then benchmark more exotic approaches against that. It's very good to use a framework like I present here (true walk forward, custom evaluation metrics) to get a more accurate idea of how complexity helps/harms performance

1 o Reply ↗

D

David Toohey

8 years ago

Great series - just a few simple edits are required: all of the hyperlinks to previous articles in the series link to this current article instead.

o o Reply ↗

C

Chad Gray → David Toohey

8 years ago

Hi David, thanks for pointing this out. I'm mobile right now but will try to fix next time I'm able to re-publish.

o o Reply ↗



alphascientist Mod

8 years ago

Thanks Dustin - glad you're enjoying them!

o o Reply ↗



Dustin Espersen

8 years ago

Fantastic series Chad! I'm really enjoying reading through these!

o o Reply ↗

[Subscribe](#)

[Privacy](#)

[Do Not Sell My Data](#)

Recent Posts

- [Reddit for Fun and Profit \[part 2\]](#)
- [Reddit for Fun and Profit \[part 1\]](#)
- [COVID By The Numbers](#)
- [Contribute to The Alpha Scientist blog!](#)
- [Stock Prediction with ML: Ensemble Modeling](#)

Tags

[data](#), [web scraping](#), [COVID](#), [administrative](#), [tutorial](#), [modeling](#), [keras](#), [ETFs](#), [market anomaly](#), [feature engineering](#), [pandas](#), [seasonality](#), [SPY](#)

Receive email notifications for future posts

Email Address

First Name

Subscribe

Social

- [RSS](#)
- [Atom](#)
- [Linkedin](#)
- [Twitter](#)

Latest Tweets

[Tweets by data2alpha](#)

- Status updating...

Follow @data2alpha

Blogroll

- [Quantocracy](#)
- [Quant-at-Risk](#)
- [Robot Wealth](#)
- [QuantStart](#)
- [Simon Ouellette](#)
- [Marcos Lopez de Prado](#)
- [Financial Hacker](#)
- [Knoema](#)
- [AlternativeData.org](#)
- [Eagle Alpha](#)
- [Neudata](#)
- [Cloudquant](#)
- [Quiver Quant](#)
- [Real Vision](#)
- [Cryptoquant](#)
- [Glassnode](#)
- [Diffbot](#)
- [TheGraph](#)
- [Mnemonic](#)
- [Yewno](#)

- [Yipit](#)